Lógica Matemática

Gabriel Martínez González

1 de marzo de 2026

Índice general

Capítulo 1 Lógica proposicional **2**
1.1 Lenguaje de la lógica proposicional 2
1.2 Semantica 2
1.3 Sintaxis 3
1.4 Sintaxis 5
1.5 Semantica 8

Capítulo 2 Lógica de primer orden **9**
2.1 El cálculo deductivo 9
2.1.1 Preliminares sintácticos 9
2.2 Modelos de la Lógica de Primer Orden (semántica) 14

Capítulo 3 Computabilidad **21**
3.1 Teoría de la computabilidad 21
3.2 Problema de la detención 23
3.3 Computabilidad y lógica 27
3.4 Segundo teorema de incompletitud 34

Capítulo 1

Lógica proposicional

1.1. Lenguaje de la lógica proposicional

Para empezar, consideremos el concepto intuitivo de alfabeto:

$$\text{alfabeto} = \begin{cases} \text{variables} & p_1, p_2, \cdots, p_n, \cdots \\ \text{conectivas lógicas} & \neg, \Longrightarrow \end{cases}$$

Definición 1. *(formulas)*
El conjunto de formulas de la lógica proposicional se define mediante las cláusuras:

- Para cada n, P_n es formula (formula atómica)
- Si φ y ψ son formulas, entonces
$$\neg\varphi \text{ y } \Longrightarrow \varphi\psi$$
también lo son.

1.2. Semantica

Definición 2. *(modelos)*
Un módelo de la lógica proposicional es una función

$$m : \{p_n | n \in \mathbb{N}\} \longrightarrow \{V, F\}$$

tal que

- $\overline{m}(p_n) = m(p_n)$, para cada $n \in \mathbb{N}$
- $\overline{m}(\neg\varphi) = V$ si, y sólo si $\overline{m}(\varphi) = F$
- $\overline{m}(\Longrightarrow \varphi\psi) = F$ si, y sólo si $\overline{m}(\varphi) = V$ y $\overline{m}(\psi) = F$

Definición 3. *(satisfacción)*
Dado un modelo m y una formula φ, diremos que m satisface φ, simbolizado $m \vDash \varphi$, si $\overline{m}(\varphi) = V$. Si Σ es un conjunto de formulas, entonces $m \vDash \Sigma$ si para cada $\varphi \in \Sigma$, $m \vDash \varphi$.

Lema 4. *Sean m_1 y m_2 dos modelos, sea Σ un conjunto de formulas y sea*

$$W = \{p_k | p_k \textit{ aparece en algún } \varphi \in \Sigma\}.$$

Si $m_1|_W = m_2|_W$, entonces para cada $\varphi \in \Sigma$,

$$\overline{m_1}(\varphi) = \overline{m_2}(\varphi)$$

$$m_1 \vDash \varphi \text{ si, y sólo si } m_2 \vDash \varphi$$

Definición 5. *(consecuencias lógicas)*
Sea φ una formula y Σ un conjunto de formulas.

- φ es una consecuencia lógica de Σ, escrito $\Sigma \vDash \varphi$, si para cada modelo m tal que $m \vDash \Sigma$, se cumple que $m \vDash \varphi$.
- φ es una tautología si
$$\varnothing \vDash \varphi \text{ (comúnmente } \vDash \varphi)$$
- φ es una contradicción si
$$\vDash \neg\varphi$$

Observación 6. A partir de ahora, tenemos:

$$\varphi \wedge \psi \text{ significa } \neg(\varphi \Longrightarrow \neg\psi)$$

y

$$\varphi \vee \psi \text{ significa } \neg\varphi \Longrightarrow \psi$$

Teorema 7. *Si φ es una formula y $\Sigma = \{\psi_1, \ldots, \psi_n\}$ es un conjunto finito, entonces*

$$\Sigma \vDash \varphi \text{ si, y sólo si } (\psi_1 \wedge \cdots, \psi_k) \Longrightarrow \varphi \text{ es una tautología.}$$

Definición 8. *(satisfacible)*
Σ es satisfacible si existe un modelo m tal que

$$m \vDash \Sigma$$

Proposición 9. *Sea φ una formula y Σ un conjunto de formulas. Entonces,*

$$\Sigma \vDash \varphi \text{ si, y sólo si } \Sigma \cup \{\neg\varphi\} \text{ no es satisfacible.}$$

1.3. Sintaxis

Definición 10. *(axiomas de lógica proposicional)*
Planteamos la siguiente lista de axiomas lógicos:

- $\varphi \Longrightarrow (\psi \Longrightarrow \varphi)$
- $\varphi \Longrightarrow ((\psi \Longrightarrow \neg\varphi) \Longrightarrow \neg\psi)$
- $\varphi \Longrightarrow \varphi[\psi \leftrightsquigarrow \neg\neg\psi]$
- $\varphi \Longrightarrow \varphi[\psi \Longrightarrow \chi \leftrightsquigarrow \neg\chi \Longrightarrow \neg\psi]$
- $\varphi \Longrightarrow \varphi[\neg\psi \Longrightarrow \psi \leftrightsquigarrow \psi]$
- $(\varphi \Longrightarrow (\chi \Longrightarrow \psi)) \Longrightarrow ((\varphi \Longrightarrow \chi) \Longrightarrow (\varphi \Longrightarrow \psi))$

Y una regla de inferencia:

- Modus Ponens

$$\frac{\begin{array}{c}\varphi \implies \psi \\ \varphi\end{array}}{\therefore \psi}$$

Definición 11. *(demostraciones y teoremas)* Sea φ una formula y Σ un conjunto de formulas.

- Una textbfdemostración de φ a partir de Σ es una sucesión finita de formulas $(\varphi_1, \varphi_2, \ldots, \varphi_n)$, tales que:
  1. φ_n es φ
  2. Cada φ_i, satisface uno de los siguientes casos:
     - $\varphi_i \in \Sigma$.
     - φ_i es axioma lógico.
     - Existen $j, k < i$ tales que φ_j es $\varphi_k \implies \varphi_i$.
- Decimos que φ es demostrable a partir de Σ, o que φ es un **teorema** de Σ, si existe una demostración de φ a partir de Σ. Se simboliza $\Sigma \vdash \varphi$.

Definición 12. *(Más reglas de inferencia)*
Definiremos las siguientes reglas de inferencia, demostrables a partir de la definición anterior:

Modus Tollens
$$\frac{\begin{array}{c}\varphi \implies \psi \\ \neg\psi\end{array}}{\therefore \neg\varphi}$$

Adición
$$\frac{\varphi}{\therefore \varphi \vee \psi}$$

Simplificación
$$\frac{\varphi \wedge \psi}{\therefore \psi}$$

Conjunción
$$\frac{\begin{array}{c}\varphi \\ \psi\end{array}}{\therefore \varphi \wedge \psi}$$

Silogismo disjuntivo
$$\frac{\begin{array}{c}\varphi \vee \psi \\ \neg\varphi\end{array}}{\therefore \psi}$$

Doble negación 1
$$\frac{\varphi}{\therefore \varphi[\psi \leftrightsquigarrow \neg\neg\psi]}$$

Doble negación 2
$$\frac{\varphi}{\therefore \varphi[\neg\neg\psi \leftrightsquigarrow \psi]}$$

Transposición
$$\frac{\varphi \implies \psi}{\therefore \neg\psi \implies \neg\varphi}$$

Ley de Morgan ($\wedge$)
$$\frac{\neg(\varphi \wedge \psi)}{\therefore \neg\varphi \vee \neg\psi}$$

Ley de Morgan ($\vee$)
$$\frac{\neg(\varphi \vee \psi)}{\therefore \neg\varphi \wedge \neg\psi}$$

Lema 13.

$$(\forall\varphi)(\forall\Sigma)(\Sigma \vdash \varphi \implies \varphi)$$

Teorema 14. *(de deducción)(demostración condicional)*
Sean φ, ψ formulas y Σ un conjunto de formulas. Entonces,

$$\Sigma \cup \{\varphi\} \vdash \psi \iff \Sigma \vdash (\varphi \implies \psi)$$

Teorema 15. *(reducción al absurdo)*
Sea φ una formula y sea Σ un conjunto de formulas. Entonces,

$$\Sigma \cup \{\neg\varphi\} \vdash \psi \wedge \neg\psi \implies \Sigma \vdash \varphi$$

Teorema 16. *(demostración por casos)*
Sean φ, ψ formulas y sea Σ un conjunto de formulas. Si $\Sigma \cup \{\varphi\} \vdash \psi$ y $\Sigma \cup \{\neg\varphi\} \vdash \psi$, entonces $\Sigma \vdash \psi$.

Definición 17. *(consistencia)*
Un conjunto de formulas Σ es inconsistent si $\Sigma \vdash \psi \wedge \neg\psi$, para alguna formula *psi*. En caso contrario, Σ es consistente.

Teorema 18. *(explosividad de la lógica proposicional)*
Sea Σ un conjunto de formulas. Entonces, Σ es inconsistente si, y sólo si, $\Sigma \vdash \varphi$, para cada formula φ.

Corolario 19. *(criterio de consistencia existencia de proposiciones)*
Si Σ es un conjunto de formulas, entonces Σ es consistente si, y sólo si existe alguna formula φ tal que $\Sigma \nvdash \varphi$.

Proposición 20. *(criterio de consistencia por negación de proposiciones)*
Sea φ una formula y Σ un conjunto de formulas. Entonces,

$$\Sigma \vdash \varphi \Longleftrightarrow \Sigma \cup \{\neg\varphi\} \text{ es inconsistente};$$

analogamente,

$$\Sigma \nvdash \varphi \Longleftrightarrow \Sigma \cup \{\neg\varphi\} \text{ es consistente.}$$

Teorema 21. *(de correctud)*
Para cada formula φ y cada conjunto de formulas Σ, tenemos que

$$\Sigma \vdash \varphi \implies \Sigma \vDash \varphi.$$

Lema 22. *(de agregar una proposición a un conjunto consistente)*
Sea φ una formula y Σ un conjunto de formulas consistente. Entonces, o bien $\Sigma \cup \{\varphi\}$, o bien $\Sigma \cup \{\neg\varphi\}$ es un conjunto consistente.

Teorema 23. *(de completud)*
Si Σ es consistente, entonces Σ es satisfacible.

Teorema 24. *(de completud) (versión 2)*
Si $\Sigma \vDash \varphi$, entonces $\Sigma \vdash \varphi$.

Teorema 25. *(de completud) (versión 3)*
Σ es consistente si, y sólo si Σ es satisfacible.

Teorema 26. *(de completud) (versión 4)*

$$\Sigma \vDash \varphi \Longleftrightarrow \Sigma \vdash \varphi.$$

Teorema 27. *(compacidad de la lógica proposicional)*
Sea φ una formula y Σ un conjunto de formulas. Entonces,

$$\Sigma \vDash \varphi \Longleftrightarrow (\exists \psi_1, \ldots, \psi_n \in \Sigma)(\{\psi_1, \ldots, \psi_n\} \vDash \varphi).$$

1.4. Sintaxis

Definición 28. *(lenguaje)*
Un lengujae de primer orden consta de un alfabeto como sigue:

1. variables: $v_1, \ldots, v_n, \ldots$
2. conectivas lógica: $\neg$, $\implies$
3. cuantificador: $\forall$
4. igualdad: $=$
5. símbolos de constante: $c_1, c_2, \ldots$

6. símbolos de función: $F_1, F_2, \ldots$

7. símbolos de relación: $R_1, R_2, \ldots$

donde los puntos 1 a 4 son llamados símbolos lógicos y los puntos 5 a 8 son símbolos no lógicos, de los cuales los de función y los de relación tienen asociado un número que denota su *aridad*. Así mismo, el lenguaje se dentoa por:

$$\mathcal{L} = \{c_1, \ldots, F_1, \ldots, R_1, \ldots\}.$$

Definición 29. *(término)*
Sea $\mathcal{L} = \{c_1, \ldots, F_1, \ldots, R_1, \ldots\}$ un lenguaje de primer orden. Un termino de $\mathcal{L}$ es una sucesión finita de símbolos del alfabeto de $\mathcal{L}$ qque proviene de las cláusuras:

1. Para cada n, v_n es término.

2. Para cada i, c_i es término.

3. Si F_i es un símbolo de función de aridad k y $t_1, \ldots, t_k$ son términos, entonces $F_i t_1 \ldots t_k$ es un término.

Definición 30. *(fórmula)*
Sea $\mathcal{L}$ un lenguaje de primer orden. Una fórmula es una sucesión finita de símbolos del alfabeto de $\mathcal{L}$, que proviene de algunas de las siguientes cláusuras:

1. Si t_1, t_2 son terminos, entonces $= t_1 t_2$ es una fórmula.

2. Si R_i es un símbolo de aridad k, y $t_1, \ldots, t_k$ son términos, entonces $R_i t_1, \ldots t_k$ es una fórmula.

3. Si φ es fórmula, $\neg\varphi$ es también fórmula.

4. Si φ, ψ son fórmulas, $\implies \varphi\psi$ también es fórmula.

5. Si φ es una fórmula y v_i es una variable, $\forall v_i \varphi$ también lo es.

Definición 31. *(sustitución)*
La expresión

$$\varphi\,[t/x]$$

es el resultado de sustituir cada aparición libre de x con el termino t. Así mismo, t es sustituible por x en φ si cada aparición de t en $\varphi\,[t/x]$ contiene únicamente variables libres.

Definición 32. *(axiomas de primer orden)*
Planteamos la siguiente lista de axiomas lógicos:

0. Axiomas de lógica proposicional,

es decir,

- $\varphi \implies (\psi \implies \varphi)$;
- $\varphi \implies ((\psi \implies \neg\varphi) \implies \neg\psi)$;
- $\varphi \implies \varphi[\psi \leftrightsquigarrow \neg\neg\psi]$;
- $\varphi \implies \varphi[\psi \implies \chi \leftrightsquigarrow \neg\chi \implies \neg\psi]$;
- $\varphi \implies \varphi[\neg\psi \implies \psi \leftrightsquigarrow \psi]$;
- $(\varphi \implies (\chi \implies \psi)) \implies ((\varphi \implies \chi) \implies (\varphi \implies \psi))$;

Y cualquier generalización de los siguientes enunciados:

1. $(\forall x)(\varphi \implies \psi) \implies (\forall x\varphi \implies \forall x\psi)$;
2. $\varphi \implies (\forall x\varphi)$, si x no aparece libre en φ;
3. $x = x$;
4. $x = y \implies (\varphi \implies \varphi')$, si φ es atómica y donde φ' resulta de reemplazar algunas apariciones de x por y en φ;
5. $\forall x\varphi \implies (\varphi\,[t/x])$ si t es un término, sustituible por x en φ.

Definición 33. *(demostración y demostrabilidad)*
Sea Σ un conjunto de formulas.

1. Una demostración de φ a partir de Σ es una sucesión finita de fórmulas $(\varphi_1, \ldots, \varphi_n)$ tal que φ_i es elemento de Σ, axioma lógico o existen $j, k < i$ tales que φ_k es la fórmula $\varphi_j \implies \varphi_i$.
2. $\Sigma \vdash \varphi$ significa que existe una demostración formal de φ a partir de Σ.
3. Σ es consistente si no hay ψ tal que
$$\Sigma \vdash \psi \wedge \neg\psi.$$

Definición 34. *(reglas de inferencía de primer orden)*
Tenemos las siguientes reglas de inferencia:

0. Todas las reglas de inferencía lógica proposicional;
1. Instantación universal: siempre que t sea sustituible,
$$\frac{\forall x\varphi}{\therefore \varphi[t/x]}$$
2. Generalización existencial: siempre que t sea sustituible,
$$\frac{\varphi[t/x]}{\therefore \exists x\varphi}$$
3. Generalización universal: siempre que x no aparezca libre en alguna premisa,
$$\frac{\varphi}{\therefore \forall x\varphi}$$
4. Instantación existencial: en donde u no so aparece libre en ninguna premisa, en la formula $\exists x\varphi$, ni en ψ,
$$\frac{\begin{array}{c}\exists y\varphi \\ \varphi[u/y] \text{ (como premisa extra)} \\ \vdots \\ \psi \text{ (que ya no contiene libre a u)}\end{array}}{\therefore \psi}$$
5. Cambio de variable: si z no aparece libre en φ ni en ninguna premisa,
$$\frac{\forall x\varphi}{\therefore \forall z\varphi[z/x]}$$

6. Sustitución: si φ' resulta de reemplazar algunas apariciones de x por y en φ,

$$\frac{\begin{array}{c} x = y \\ \varphi \end{array}}{\therefore \varphi'}$$

Lema 35. *(cambio de variable)*
$\Sigma \cup \{\forall x\varphi\} \vdash \forall z\varphi[z/x]$, siempre que z sea sustituible por x en φ y z no aparezca libre en ningún elemento de $\Sigma \cup \{\forall x\varphi\}$.

Teorema 36. *(generalización universal)*
Sea φ una fórmula y Σ un conjunto de formula. Si $\Sigma \vdash \varphi$, y x no aparece libre en ningún elemento de Σ, entonces $\Sigma \vdash \forall x\varphi$.

Teorema 37. *(instantación universal)*
Si $\Sigma \cup \{\varphi[w/x]\} \vdash \psi$, entonces $\Sigma \cup \{\exists x\varphi\} \vdash \psi$ siempre y cuando w no aparezca libre en ningún elemento de Σ, ni en ψ, ni en $\exists x\varphi$; ni x aparezca libre en ningún elemento de Σ.

1.5. Semantica

Capítulo 2

Lógica de primer orden

2.1. El cálculo deductivo

A continuación explicaremos el cálculo deductivo que corresponde a la lógica de primer orden. Es decir, haremos explícitos los axiomas lógicos y reglas de inferencia que nos permiten llevar a cabo demostraciones formales en lenguajes de primer orden.

2.1.1. Preliminares sintácticos

Dado que este desarrollo es completamente sintáctico, debemos comenzar por definir otra noción completamente sintáctica: la noción de, cuando una variable aparece en una fórmula, si aparece de manera libre o ligada.

Observemos lo siguiente: consideremos el conjunto de las variables de nuestro lenguaje, el cuál denotaremos por *var*, el conjunto de términos de nuestro lenguaje, el cuál denotaremos por *Term*, el conjunto de tuplas con entradas $\{0,1\} \cup \text{var}$, denotado por *tup*, y consideremos la operación de concatenación de tuplas (es decir, pegar tuplas y volverlas una más grande) mediante el símbolo $\oplus$. Observemos entonces la función:

$$\text{Fr: Term} \longrightarrow \text{tup}$$

Definida por las siguientes reglas:
$\text{Fr}(C_k) = \emptyset$ (es decir, la tupla vacía),
$\text{Fr}(V_k) = (V_k, 1), \qquad \text{Fr}(F_k t_1 \cdots t_n) = \text{Fr}(t_1) \oplus \cdots \oplus \text{Fr}(t_n).$

Esta función simplemente recoge las variables que aparecen en un cierto termino, en orden y con repeticiones, y pone un 1 inmediatamente después de cada una. Ahora, consideremos el conjunto de formulas, al cuál denotaremos por *Form*. Consideremos la siguiente función:

$$\text{Op:tup}\times\text{var}\longrightarrow\text{tup}$$

Dada por: $\text{Op}((x_\alpha)_{\alpha\in[[1,k]]}, v_i) = (y_\alpha)_{\alpha\in[[1,k]]}$, donde:

$$y_\alpha = \begin{cases} 0, x_{\alpha-1} = v_i; \\ x_\alpha, \text{en otro caso.} \end{cases}$$

Ahora, consideremos la siguiente función:

$$\text{Free:Form} \longrightarrow \text{tup}$$

Definida por:

- Free($t_1 t_2$)=Free(t_1)$\oplus$ Free(t_2)
- Free($R_k t_1, \ldots, t_n$) =Free(t_1) $\oplus \cdots \oplus$ Free(t_n)
- Free ($\neg\varphi$) = Free (φ)
- Free ($\Rightarrow \varphi\psi$)=Free(φ)$\oplus$Free(ψ)
- Free($\forall V_k \varphi$)= Op(Free(φ),V_k).

Definición 38. Una variable v_i está ligada en una formula φ si en la tupla Free(φ), aparece la variable v_i al menos una vez y el termino a la derecha de cada aparición de v_i es un 0.

Definición 39. Un **enunciado** es una fórmula φ sin variables libres.

Definición 40. Decimos que una generalización de la formula φ sobre la variable x es $\forall x \varphi$.

Ahora: dado un término t, y una variable x, explicar la sustitución $\varphi[t/x]$ (definida por inducción sobre la fórmula φ: si φ es atómica, simplemente poner t en lugar de x; después, $(\Rightarrow \varphi\psi)[t/x]$ es la fórmula $\Rightarrow \varphi[t/x]\psi[t/x]$, $(\neg\varphi)[t/x]$ es la fórmula $\neg\varphi[t/x]$, y $(\forall y\ \varphi)[t/x]$ es la fórmula $\forall y\ \varphi[t/x]$, si y no es la variable x, y $\forall x\ \varphi$ en caso contrario).

Ejemplos de captura: calcular $\varphi[y/x]$ en los casos cuando φ es la fórmula:

1. $(Qx \Rightarrow \forall x\ Px)$,
2. $\neg\forall z\ x = z$,
3. $\neg\forall y\ x = y$.

En el tercer caso ocurre la "captura" de la variable y, esta es la situación a evitar. Por ello definimos lo que significa que un término t sea sustituíble por una variable x en φ, desde luego por inducción sobre φ: si φ es atómica, entonces de cajón t es sustituíble por x en φ. Similarmente, t es sustituíble por x en $\neg\varphi$ si y sólo si lo es en φ, y en $\Rightarrow \varphi\psi$ si y sólo si lo es en φ y en ψ. Finalmnete, t es sustituíble por x en $\forall y\ \varphi$ si y sólo si: o bien x no aparece libre en $\forall y\ \varphi$, o bien y no aparece en t y t es sustituíble por x en φ.

Ahora sí, procederemos a enunciar las bases de nuestro sistema de deducción lógica. Al igual que en el caso de la lógica proposicional, nuestra única regla de inferencia será el *Modus Ponens*, lo cual se formalizará cuando enunciemos la definición de "demostración formal". En cuanto a los axiomas lógicos, son los siguientes.

Definición 41. El conjunto de axiomas lógicos será el que contiene a cualquier generalización de cualquiera de los siguientes tipos de fórmulas:

Axioma 0: Todos los axiomas de la lógica proposicional,

Axioma 1: $\forall x\ (\varphi \Rightarrow \psi) \Rightarrow (\forall x\ \varphi \Rightarrow \forall x\ \psi)$,

Axioma 2: $\varphi \Rightarrow \forall x\ \varphi$, siempre que x no aparezca libre en φ,

Axioma 3: $\forall x\ \varphi \Rightarrow \varphi[t/x]$, siempre que t sea sustituíble por x en φ,

Axioma 4: $x = x$,

Axioma 5: $x = y \Rightarrow (\varphi \Rightarrow \varphi')$, siempre que φ sea una fórmula atómica, y φ' resulte de sustituír algunas apariciones de x por y en la fórmula φ.

Definición 42. Sea φ una fórmula, y sea Σ un conjunto de fórmulas.

1. Una **demostración** de φ a partir de Σ es una sucesión finita de fórmulas, $(\varphi_1, \ldots, \varphi_n)$, tal que φ_n es la fórmula φ, y, para cada i, se cumple una de las siguientes opciones:

a) o bien $\varphi_i \in \Sigma$,

b) o bien φ_i es un axioma lógico,

c) o bien existen $j, k < i$ tales que φ_k es la fórmula $\varphi_j \Rightarrow \varphi_i$.

2. Diremos que φ es **demostrable a partir de** Σ, lo cual se simbolizará como $\Sigma \vdash \varphi$, si existe una demostración formal de φ a partir de Σ.

A continuación procederemos, al igual que fue en el caso de la lógica proposicional, a desarrollar algunas "reglas de inferencia derivadas" que nos permitirán hacer más cortas nuestras demostraciones formales. Serán esencialmente cinco reglas: instanciación ("quitar el cuantificador") universal (para todo) y existencial (existe), y generalización ("poner el cuantificador") universal y existencial. Además, introduciremos una regla más para cambios de variable, reflejando el hecho de que la variable utilizada en una fórmula cuantificada "es muda".

Instanciación Universal

$$\frac{\forall x\ \varphi}{\therefore \quad \varphi[t/x],} \quad \text{siempre que } t \text{ sea sustituíble por } x \text{ en } \varphi.$$

Esta regla queda justificada por la siguiente demostración formal:

1	$\forall x\ \varphi$	Premisa
2	$\forall x\ \varphi \Rightarrow \varphi[t/x]$	Axioma 3
3	$\varphi[t/x]$	Modus Ponens 2, 1

Generalización existencial

$$\frac{\varphi[t/x]}{\therefore \quad \exists x\ \varphi,} \quad \text{siempre que } t \text{ sea sustituíble por } x \text{ en } \varphi.$$

Esta regla queda justificada por la siguiente demostración formal:

1	$\varphi[t/x]$	Premisa
2	$\forall x\ \neg\varphi \Rightarrow \neg\varphi[t/x]$	Axioma 3
3	$\neg\neg\varphi[t/x]$	Doble Negación, 1
4	$\neg\forall x\ \neg\varphi$	Modus Tollens 2, 3

Lo cual, desde luego, finaliza nuestra demostración, dado que la última línea de la misma es precisamente la fórmula que abreviamos como $\exists x\ \varphi$.

Ejercicios

Demuestre que existe una relación de demostrabilidad como se muestra a continuación.

1. $\{\forall x\ Px \Rightarrow Qx\} \vdash Pc \Rightarrow (\forall y\ (Qy \Rightarrow Sy) \Rightarrow Sc)$,
2. $\{\forall x\ (Px \Rightarrow \forall y\ (Qy \Rightarrow Sy))\} \vdash \forall x\ Px \Rightarrow \forall y(Qy \Rightarrow Sy)$,
3. $\{\exists x\ Px \Rightarrow \exists y\ Qy\} \vdash \exists x\ (Px \Rightarrow \exists y\ Qy)$.

Generalización Universal

$$\frac{\varphi}{\therefore \quad \forall x\ \varphi,} \quad \text{siempre que } x \text{ no aparezca libre en ninguna premisa.}$$

Para justificar esta regla de inferencia, habremos de echarnos el siguiente metateorema altamente no trivial:

Teorema 43. *Seas φ una fórmula, y sea Σ un conjunto de fórmulas. Si $\Sigma \vdash \varphi$ y x no aparece libre en ningún elemento de Σ, entonces $\Sigma \vdash \forall x\ \varphi$.*

Demostración: Considere el conjunto

$$\Gamma = \{\psi \big| \Sigma \vdash \forall x\ \psi\}.$$

Entonces, el metateorema que queremos demostrar es equivalente a pedir que $\varphi \in \Gamma$. Nuestra estrategia será demostrar que Γ incluye a todos los axiomas lógicos, a todos los elementos de Σ, y además es cerrado bajo Modus Ponens, en el sentido de que si $\psi \Rightarrow \xi \in \Gamma$ y $\psi \in \Gamma$, entonces $\xi \in \Gamma$. Una vez hecho esto, es inmediato verificar que, siempre que $\Sigma \vdash \psi$, entonces $\psi \in \Gamma$. En particular $\varphi \in \Gamma$, y habremos terminado. Procedamos entonces a demostrar lo anunciado.

En primer lugar, consideremos el caso cuando ψ es un axioma lógico. Entonces, la fórmula $\forall x\ \psi$ también es un axioma lógico (por definición, cualquier generalización de un axioma lógico es también un axioma lógico). Por tanto $\Sigma \vdash \forall x\ \psi$, de modo que $\psi \in \Gamma$.

Pasemos ahora al caso cuando $\psi \in \Sigma$. Entonces, nuestra hipótesis (de que la variable x no aparece libre en ningún elemento de Σ) implica en particular que la variable x no aparece libre en ψ, de modo que $\psi \Rightarrow \forall x\ \psi$ es una instancia del Axioma 2. Así, la siguiente demostración formal:

1	ψ	Premisa
2	$\psi \Rightarrow \forall x\ \psi$	Axioma 2,
3	$\forall x\ \psi$	Modus Ponens 2, 1

muestra que $\Sigma \vdash \forall x\ \psi$, de modo que $\psi \in \Gamma$.

Finalmente, supongamos que $\psi \Rightarrow \xi \in \Gamma$ y $\psi \in \Gamma$. Esto significa que $\Sigma \vdash \forall x\ (\psi \Rightarrow \xi)$ y $\Sigma \vdash \forall x\ \psi$. Así, podemos considerar la siguiente demostración formal:

1	$\forall x\ (\psi \Rightarrow \xi)$	Demostrable a partir de Σ
2	$\forall x\ (\psi \Rightarrow \xi) \Rightarrow (\forall x\ \psi \Rightarrow \forall x\ \xi)$	Axioma 1,
3	$\forall x\ \psi \Rightarrow \forall x\ \xi$	Modus Ponens 2, 1
4	$\forall x\ \psi$	Demostrable a partir de Σ
5	$\forall x\ \xi$	Modus Ponens 3, 4

De esta forma, tenemos que $\Sigma \vdash \forall x\ \xi$, de modo que $\xi \in \Gamma$. Con esto, por lo explicado arriba, la demostración queda terminada. $\square$

Cambio de Variable

$$\frac{\forall x\ \varphi}{\therefore \quad \forall z\ \varphi[z/x],} \quad \begin{array}{l}\text{siempre que } z \text{ no aparezca libre en ninguna premisa} \\ \text{y sea sustituíble por } x \text{ en } \varphi.\end{array}$$

Esta regla queda justificada por la siguiente demostración formal:

1	$\forall x\ \varphi$	Premisa
2	$\varphi[z/x]$	Instanciación Universal, 1
3	$\forall z\ \varphi[z/x]$	Generalización Universal, 2

Instanciación Existencial

El contenido de esta regla derivada se encuentra en el siguiente metateorema.

Teorema 44. *Sean φ, ψ dos fórmulas, y sea Σ un conjunto de fórmulas. Sea w una variable que no aparece libre en ningún elemento de Σ ni tampoco en las fórmulas $\exists x\ \varphi$ ni ψ. Suponga además que la variable x no aparece libre en ningún elemento de Σ. Si $\Sigma \cup \{\varphi[w/x]\} \vdash \psi$, entonces $\Sigma \cup \{\exists x\ \varphi\} \vdash \psi$.*

Demostración: La hipótesis, junto con el metateorema de deducción, implican que $\Sigma \vdash \varphi[w/x] \Rightarrow \psi$. Entonces, considere la siguiente demostración por contradicción a partir de $\Sigma \cup \{\exists x\ \varphi\}$.

$\longrightarrow$	1	$\neg\psi$	Suposición extra
	2	$\varphi[w/x] \Rightarrow \psi$	Demostrable a partir de Σ.
	3	$\neg\varphi[w/x]$	Modus Ponens 2, 1
	4	$\forall w\ \neg\varphi[w/x]$	Generalización Universal, 3
	5	$\forall x\ \neg\varphi$	Cambio de Variable, 4
	6	$\neg\forall x\ \neg\varphi$	Premisa
	7	$\forall x\ \neg\varphi \wedge \neg\forall x\ \neg\varphi$	Conjunción 5, 6
	8	ψ	Demostración por contradicción 1-7

Recuerde que $\exists x\ \varphi$ es simplemente una abreviación de la fórmula $\neg\forall x\ \neg\varphi$, lo que justifica el renglón 6 de la demostración. La generalización universal del renglón 4 se puede realizar gracias a que la variable w, por hipótesis, no aparece libre ni en ningún elemento de Σ, ni en $\neg\psi$, ni en $\exists x\ \varphi$. El cambio de variable del renglón 5 está justificado por el hecho de que x no aparece libre en ningún elemento de Σ. $\square$

En la práctica, el teorema previo se utilizará como sigue: siempre que tengamos disponible una premisa de la forma $\exists x\ \varphi$, nos permitiremos escribir a continuación la fórmula $\varphi[w/x]$ como una premisa extra (poniendo cuidado en utiliz ar una variable w que no aparezca libre en ninguna premisa). Procedemos con la demostración hasta encontrar la fórmula ψ deseada, en la cual ya no aparezca libre la variable w; en ese momento, "cerramos" nuestra premisa extra y a continuación escribimos la fórmula ψ, con pleno conocimiento de que esa fórmula de hecho puede ser demostrada a partir de nuestras premisas.

Ejercicios

Demuestre las siguientes relaciones de demostrabilidad.

1. $\{\forall x\ (Px \Rightarrow Qx)\} \vdash \forall x\ (Sx \Rightarrow (\forall y\ (Sy \Rightarrow Py) \Rightarrow Qx))$,
2. $\{\forall x\ (Px \Rightarrow Qx), \forall x\ (Sx \Rightarrow Tx)\} \vdash \forall x\ (Qx \Rightarrow Sx) \Rightarrow \forall y\ (Py \Rightarrow Ty)$,
3. $\{\exists x\ Px \Rightarrow \forall y\ ((Py \vee Qy) \Rightarrow Sy), \exists x\ Px \wedge \exists x\ Sx\} \vdash \exists x\ (Px \wedge Sx)$,
4. $\{\exists x\ Px \Rightarrow \forall y\ (Qy \Rightarrow Sy)\} \vdash \exists x\ (Px \wedge Qx) \Rightarrow \exists y\ (Py \wedge Sy)$,
5. $\{\forall x\ \exists y\ (Px \vee Qy)\} \vdash \forall x\ Px \vee \exists y\ Qy$,
6. $\{\exists x\ Px \vee \forall y\ (Py \Rightarrow Qy), \forall x\ (Sx \Rightarrow \neg Px)\} \vdash \forall x\ (Px \Rightarrow Sx) \Rightarrow \forall y\ (Py \Rightarrow Qy)$,
7. $\{\exists\ Px \vee \exists y\ Qy, \forall x\ (Px \Rightarrow Qx)\} \vdash \exists y\ Qy$.

Finalmente, haremos explícito cómo utilizar los últimos 2 axiomas.

Identidad

$$\frac{}{\therefore\quad t = t,}\quad \text{en donde } t \text{ es cualquier término.}$$

Esta regla queda justificada por la siguiente demostración formal, dado un término t:

1	$\forall x\ x = x$	Axioma 4
2	$t = t$	Instanciación Universal, 1

Sustitución

$$\begin{array}{rl} \varphi & \text{, en donde } \varphi \text{ es una fórmula atómica} \\ x = y & \\ \hline \therefore \quad \varphi', & \text{en donde } \varphi' \text{ resulta de sustituir algunas instancias de } x \text{ por } y \text{ en } \varphi. \end{array}$$

Esta regla queda justificada por la siguiente demostración formal:

1	$x = y$	Premisa
2	φ	Premisa
3	$x = y \Rightarrow (\varphi \Rightarrow \varphi')$	Axioma 5
4	$\varphi \Rightarrow \varphi'$	Modus Ponens 3, 1
5	φ'	Modus Ponens 4, 2

Ejercicios

Demuestre las siguientes relaciones de demostrabilidad.

1. $\{x = y\} \vdash y = x$,
2. $\{x = y, y = z\} \vdash x = z$,
3. $\{x = z, y = w\} \vdash P(x, y) \Rightarrow P(z, w)$,
4. $\{x = z, y = w\} \vdash F(x, y) = F(z, w)$.

2.2. Modelos de la Lógica de Primer Orden (semántica)

Def: Sea $\mathcal{L} = \{\mathcal{C}_i, \mathcal{R}_i, \mathcal{F}_i\}$ un lenguaje de primer orden. Una **$\mathcal{L}$-estructura** o un **$\mathcal{L}$-módelo** es una tupla
$\mathcal{M} = (\mathcal{A}, \mathcal{C}_i, \mathcal{R}_i, \mathcal{F}_i)$ tales que:

1. $\mathcal{A}$ es un conjunto no vacío.
2. Cada $\mathcal{C}_i \in \mathcal{A}$
3. Si $\mathcal{R}_i$ es un simbolo de relación n-ario, entonces $\mathcal{R}_i \subseteq \mathcal{A}^n$
4. Si $\mathcal{F}_i$ es un simbolo de operación n-aria, entonces $\mathcal{F}_i : \mathcal{A}^n \longrightarrow \mathcal{A}$

Ejemplo:

- $\mathcal{L} = \{\ *, ()^{-1}, e\}$
- $\mathcal{L} = \{+, \cdot, 1, 0\}$
- $\mathcal{L} = \{+, \cdot, \mathcal{S}, 1, \leq\}$

Satisfacción de fórmulas:

Def: Sea $\mathcal{L} = \{\mathcal{C}_i, \mathcal{F}_i, \mathcal{R}_i\}$ un lenguaje de primer orden y sea $\mathcal{M} = \{\mathcal{A}_i, \mathcal{C}_i, \mathcal{R}_i, \mathcal{F}_i$ una $\mathcal{L}$-estructura:

1. **Una interpretación:** es una función ξ:var $\longrightarrow$ A
2. **Dada una interpretación C**, existe una única extensión $\tau : term \longrightarrow A$ dada por:

$$\tau \begin{cases} \imath(x) = \tau(x) \\ \imath(C_i) = C_i \end{cases}$$

3. Definimos: $\mathcal{M} \vDash \varphi[\xi]$, i.e "$\mathcal{M}$ satisface φ relativa a ξ"por recursión sobre φ:

$$\begin{cases} \mathcal{M} \vDash (= t_1 t_2)[\iota] \\ \text{si y sólo si } \iota(t_1) \text{ es } \iota(t_2) \\ \mathcal{M} \vDash (R_1 t_1, ..., t_n)[\iota] \\ \text{si y sólo si } < \tau(t_1), ..., \tau(t_n) > \in \mathcal{R}_i \end{cases}$$

$$\mathcal{M} \vDash \neg\varphi[\iota] \text{ si y sólo si } \mathcal{M} \nvDash \varphi[\iota]$$
$$\mathcal{M} \vDash (\varphi \Rightarrow \psi)[\iota]$$
$$\text{si y sólo si o bien } \mathcal{M} \nvDash \varphi[\iota] \text{ o bien } \mathcal{M} \vDash \psi[\iota]$$

$\mathcal{M} \vDash (\forall x \varphi)[\iota]$ si y sólo si para cada a∈A definimos ι_a como la interpretación que es igual a ι excepto que a $\iota_a(x) = a$:

$$\mathcal{M} \vDash \varphi[\iota_a]$$

Lema: Si φ únicamente tiene libres a las variables $x_1, ..., x_n$ y $\iota, \mathcal{V}$ son dos interpretaciones tales que
$\iota(x_i) = \mathcal{V}(x_i)$ para toda $i \in \{1, ...n\}$ entonces: $\mathcal{M} \vDash \varphi[\iota]$ si y sólo si $\mathcal{M} \vDash \varphi[\mathcal{V}]$
Notación: Si φ es una fórmula, si sus variables libres están entre $x_1, ..., x_n$ y ι es una interpretación tal que $\iota(x_i) = a_i \in A$ entonces simplemente escribimos $\mathcal{M} \vDash \varphi[\alpha_1, .., \alpha_n]$.
· Si φ es un enunciado, entonces tiene sentido escribir $\mathcal{M} \vDash \varphi$ o $\mathcal{M} \nvDash \varphi$

Def: Si $\mathcal{L}$ es un lenguaje de primer orden, $\mathcal{M}$ es una $\mathcal{L}$-estructura, Γ es un conjunto de fórmulas y φ es una fórmula, entonces:

1. Decimos que $\mathcal{M} \vDash \Gamma$ si y sólo si: $\forall \psi \in \Gamma$ y $\forall$ interpretación ι $\mathcal{M} \vDash \psi[\iota]$.
2. Decimos que φ es **consecuencia lógica de** Γ si para toda $\mathcal{L}$-estructura $\mathcal{M}$ tal que $\mathcal{M} \vDash \Gamma$, se tiene que $\mathcal{M} \vDash \varphi$

$$\Gamma \vdash \varphi$$
$$\Gamma \vDash \varphi \text{ sin variables libres.}$$
$$(\mathcal{A}, \mathcal{R}_i, \mathcal{F}_i, c_i) = \mathcal{M} \vDash \varphi$$

Ejemplo: En $\{*, ()^{-1}, e\}$

$$\{(x*y)*z = x*(y*z), x*e = e, x*(x)^{-1} = e\} \vDash \forall x \forall y \ (y*x = y \Rightarrow x = e)$$

Def: Σ es **consistente** si $\Sigma \nvdash \varphi \wedge \neg\varphi$ [equivalentemente, si hay alguna φ tal que $\Sigma \forall \varphi$]
·Σ es **satsfacible** si existe una estructura $\mathcal{M}$ y una interpretación ι: var $\rightarrow$ A tales que $\mathcal{M} \vDash \Sigma[\iota]$.

Teorema: Correctud:

Sea Γ un conjunto de fórmulas y φ un enunciado. Entonces, $\Gamma \vdash \varphi$ implica que $\Gamma \vDash \varphi$
Dem: (bosquejo)

Inducción /n, con $(\varphi_1, .., \varphi_n)$ es la dem. de φ a partir de Γ.
n=1 : φ es axioma lógico, $\varphi \in \Gamma$
Paso inductivo: $(\varphi_1, ..., \varphi_n)$ y el teorema se cumple $\forall k < n$. Por hipótesis de inducción $\Gamma \vDash \varphi_i$ para $i < n$.

3 casos: $\varphi_n \in \Gamma$ $(*)$ φ_n es áxioma lógico. Algún φ_j es $\varphi_i \Rightarrow \varphi_n$ para algún i con $i, j < n$.

Basta revisar 2 casos:

1. $\Gamma \vDash \varphi$ si $\varphi \in \Gamma$ (por definición)
2. Si φ es un axioma lógico, entonces $\mathcal{M} \vDash \varphi$ para toda estructura $\mathcal{M}$.

Teorema: Completitud de Gödel:

$$\Gamma \vDash \varphi \text{ implica que } \Gamma \vdash \varphi$$

Este teorema es un corolario de:
Teorema: Γ es consistente si y sólo si es satisfacible.

Pues si $\Gamma \vDash \varphi$ entonces $\Gamma \cup \{\neg\varphi\}$ no es satisfacible.
$\therefore \Gamma \cup \{\neg\varphi\}$ es inconsistente por demostración por contradicción $\Gamma \vdash \varphi$. $\therefore$ Bata demostrar: Γ es consistente $\iff$ Γ es satisfacible.
$\Leftarrow$) Si Γ es inconsistente entonces $\Gamma \vdash \varphi \wedge \neg\varphi$. Por el teorema de correctud, $\Gamma \vDash \varphi \wedge \neg\varphi$ esto significa que si $\mathcal{M}$ es cualquier estructura, entonces $\mathcal{M} \nvDash \Gamma$ (pues es imposible que $\mathcal{M} \vDash \varphi \wedge \neg\varphi$).
$\Rightarrow$) Sea Γ un conjunto de fórmulas que es consistente. [P.D que existe $\mathcal{M}$ tal que $\mathcal{M} \vDash \Gamma$].
$\mathcal{L}$ lenguaje de primer orden.
Sea $\mathcal{L}^* = \mathcal{L} \cup \{D_i | i \in \mathfrak{N}\}$ $D_i \to$ Simbolo de cte.
Lema 1: Γ sigue siendo consistente con respecto a $\mathcal{L}^*$.

Dem: Supongamos que Γ es inconsistente en $\mathcal{L}^*$. Sin perdida de generalidad φ no contiene $D_{i's}$.

Cantidad finita de variables utilizados, digamos que entre $\{v_1, ..., v_n\}$.
Sustituyendo cada D_i en esa demostración por v_{n+i}, obtenemos una dem. de $\varphi \wedge \neg\varphi$ a partir de Γ en $\mathcal{L}$, contradicción.
Lema 2: Sea $\varphi_1, ..., \varphi_n ...$ una enumeración efectiva de las fórmulas de $\mathcal{L}$. Entonces el conjunto:

$$\Gamma \cup \{\exists x\varphi \Rightarrow \varphi[D_n/x] \mid n \in \mathfrak{N}\}$$

también es consistente.

Consistente $\Rightarrow$ Satisfacible

$\cdot$ Sea Γ un conjunto de fórmulas que es consistente en **un lenguaje $\mathcal{L}^*$.**

1. $\mathcal{L}^* = \mathcal{L} \cup \{D_n \mid n \in \mathfrak{N}\}$
2. Γ sigue siendo consistente en $\mathcal{L}^*$.
3. Enumeramos (φ_1, x_1),...,$(\varphi_, x_n)$... las parejas (fórmula, variable). Entonces, $\Gamma' = \Gamma \cup \{\exists\varphi_n \Rightarrow \varphi_n[D_n/x] \mid n \in \mathfrak{N}\}$ también es consistente.

Dem: Sea θ_n la fórmula $\exists x_n\varphi_n \Rightarrow \varphi_n[D_n/x]$
Supongamos que $\Gamma' = \Gamma \cup \{\theta_1, ..., \theta_n ...\}$ es inconsitente. Sea n tal que $n \in \mathfrak{N} \cup \{0\}$ tal que $\Gamma \cup \{\theta_1, ..., \theta_{n+1}\}$ es inconsistente pero $\Gamma \cup \{\theta_1, ..., \theta_n\}$ es constante. Entonces $(\Gamma \cup \{\theta_1, ..., \theta_n\}) \cup \{\theta_{n+1}\} \vdash$ contradicción. Por el MT de prueba indirecta.

$$\Gamma \cup \{\theta_1, ..., \theta_n\} \vdash \theta_{n+1}, \text{ donde } \theta_{n+1} : \exists x_{n+1}\varphi_{n+1} \wedge \varphi_{n+1}[D_{n+1}/x_{n+1}]$$

En particular, $\Gamma \cup \{\theta_1, ..., \theta_n\} \vdash \neg\varphi_{n+1}[D_{n+1}/x_{n+1}]$, donde no aparece D_{n+1}.
$\therefore$ Como en lema 1, podemos llevar a cabo la demostración usando una variable distinta y en vez de D_{n+1}.
De esta forma, $\Gamma \cup \{\theta_1, ... - \theta_n\} \vdash \neg\varphi_{n+1}[y/x_{n+1}]$. Al no ser libre τ.^{en} $\Gamma \cup \{\theta_1, ..., \theta_n\}$ por GU,

$$\Gamma \cup \{\theta_1, ..., \theta_n\} \vdash (\forall y)(\neg\varphi_{n+1}[y/x_{n+1}])$$
$$\Gamma \cup \{\theta_1, ..., \theta_n\} \vdash (\exists x_{n+1}\varphi_{n+1})$$

$\therefore \Gamma \cup \{\theta_1, ..., \theta_n\}$ es inconsistente lo que contradice la elección de n.
Notemos que: Si Σ es consistente y φ es cualquier fórmula, entonces o ien $\Sigma \cup \{\varphi\}$ o bien $\Sigma \cup \{\neg\varphi\}$

es consistente. Por lo tanto, al ser Γ' consitente lo podemos extender a un Γ^∞ que es maximalmente consistente (Γ^∞ es consistente) en $\mathcal{L}^*$.

Si $\varphi \notin \Gamma^\infty$ entonces $\Gamma^\infty \cup \{\varphi\}$ es inconsitente implica $\neg\varphi \in \Gamma^\infty$.

Propiedades de Γ^∞

1. Es consistente.
2. Para todo φ o bien $\varphi \in \Gamma^\infty$ o bien $\neg\varphi \in \Gamma^\infty$, no ambos.
3. $\varphi \in \Gamma^\infty$ si y sólo si $\Gamma^\infty \vdash \varphi$, si y sólo si $\neg\varphi \notin \Gamma^\infty$
4. Todas las $\exists x_n \varphi_n \Rightarrow \varphi_n[D_n/x_n] \in \Gamma^\infty$
5. Sea $Term^*$ el conjunto de términos de $\mathcal{L}^*$. Sea $\sim$ definida en $Term^*$ mediante: $t_1 \sim t_2$ si y sólo si
   $\Gamma^\infty \vdash (t_1 = t_2)$. Entonces $\sim$ es una relación de equivalencia.
6. Definimos A=$Term^*/\sim$:
   $\mathcal{M} = (\mathcal{A}, \mathcal{R}_i, \mathcal{F}_i, c_i, d_i)$ se define como:

   $$c_i = [C_i], d_i[D_i], \mathcal{F}_i([t_1], [t_2], ..., [t_n]) = [\mathcal{F}t_1t_2, ..., t_n] \text{ y } [t_1], ..., [t_n] \in \mathcal{R}_i \text{ si y sólo si } \mathcal{R}_i t_1, ..., t_n \in \Gamma^\infty$$

   · Supongamos $t_1 \sim s_1, ..., t_n \sim s_n$.
   Eso significa que: $\Gamma^\infty \vdash t_k = s_k \vdash \mathbf{x} = \mathbf{y} \Rightarrow \mathbf{f}(\mathbf{x}) = \mathbf{f}(\mathbf{y})$con esto se muestra que:

   $$\Gamma^\infty \vdash \mathcal{F}_i t_1, ..., t_n = \mathcal{F}_i s_i ....s_n \therefore \mathcal{F}_i t_1, ..., t_n \sim \mathcal{F}s_1...s_n$$

   También $\vdash x = y \Rightarrow R_x \iff R_y$ con esto se muestra que:

   $$\Gamma^\infty \vdash \mathcal{R}_i t_1, ..., t_n \text{ si y sólo si } \Gamma^\infty \vdash \mathcal{R}_i s_1...s_n$$

   $\therefore$ Las $\mathcal{F}_i, \mathcal{R}_\rangle$ están bien definidas:

   $$\mathcal{M} \text{ es una } \mathcal{L}^*\text{-estructura.}$$

7. Sea $\iota : var \to A$ dada por $\iota(v_n) = [v_n]$.
   Si φ es cualquier fórmula, entonces $\mathcal{M} \vDash \varphi[\iota]$ si y sólo si $\varphi \in \Gamma^\infty$

Dem: Lema $\tau(t) = [t]$
Inducción:

$$\tau(V_n) = [v_n]$$
$$\tau(C_n) = [c_n]$$
$$\tau(D_n) = [D_n]$$

Paso inductivo:

$$\begin{aligned}\tau(\mathcal{F}_i t_1 ... t_n) &= \mathcal{F}_i(\tau(t_1), ..., \tau(t_n)) \\ &= \mathcal{F}_i([t_1], ..., [t_n]) \\ &= [\mathcal{F}_i t_1, ..., [t_n]\end{aligned}$$

Por inducción sobre φ

$$\text{Base} \begin{cases} \varphi \text{ es } t_1 = t_2. \text{ Entonces } \mathcal{M} \vDash [\iota] \text{ si y sólo si} \\ \qquad\qquad\qquad\qquad [t_1] = \tau(t_1) = \tau(t_2) \end{cases}$$

$$\begin{gathered}\text{si y sólo si } \Gamma^\infty \vdash t_1 = t_2 \\ \text{si y sólo si } (t_1 = t_2) \in \Gamma^\infty\end{gathered}$$

φ es $\mathcal{R}_i t_1 ... t_n$. Entonces $\mathcal{M} \vDash \varphi[\iota]$

$$\begin{gathered}\text{si y sólo si } (\tau(t_1), ..., \tau(t_n) \in \mathcal{R}_i \\ \text{si y sólo si } ([t_1], ..., [t_n]) \in \mathcal{R}_i \\ \text{si y sólo si } \mathcal{R}_i t_1 ... t_n \in \Gamma^\infty\end{gathered}$$

Paso inductivo:

$$\begin{gathered}\mathcal{M} \vDash \neg\varphi[\iota] \text{ si y sólo si } \mathcal{M} \nvDash \varphi[\iota] \\ \text{si y sólo si } \varphi \notin \Gamma^\infty \\ \text{si y sólo si } \neg\varphi \in \Gamma^\infty\end{gathered}$$

$\mathcal{M} \nvDash (\varphi \Rightarrow \psi)[\iota]$ si y sólo si $\mathcal{M} \vDash \varphi[\iota]$ y $\mathcal{M} \nvDash \psi[\iota]$

$$\begin{gathered}\text{si y sólo si } \varphi \in \Gamma^\infty \ y \ \psi \notin \Gamma^\infty \\ \text{si y sólo si } \neg\varphi \in \Gamma^\infty\end{gathered}$$

$\mathcal{M} \nvDash (\varphi \Rightarrow \psi)[\iota]$ si y sólo si $\mathcal{M} \vDash \varphi[\iota]$ y $\mathcal{M} \nvDash \psi[\iota]$

$$\begin{gathered}\text{si y sólo si } \varphi \in \Gamma^\infty \ y \ \psi \notin \Gamma^\infty \\ \text{si y sólo si } \Gamma^\infty \vdash \varphi \ y \ \Gamma^\infty \vdash \neg\psi \\ \text{si y sólo si } \Gamma^\infty \vdash \neg(\varphi \Rightarrow \psi) \\ \text{si y sólo si } (\varphi \Rightarrow \psi) \notin \Gamma^\infty\end{gathered}$$

Finalmente,
$\mathcal{M} \vDash [\exists x]\varphi$ implica para algún $[t] \in A$

$$\mathcal{M} \vDash \varphi[x \rightsquigarrow [t]]$$

Lo cual implica que $\mathcal{M} \vDash \varphi[t/x][\iota]$ y esto implica que $\varphi[t/x] \in \Gamma^\infty$. Por GE, esto implica $\Gamma^\infty \vdash \exists x\varphi$ por GE lo cual implica $(\exists x)\varphi \in \Gamma^\infty$.
Reciprocamente, $(\exists x\varphi) \in \Gamma^\infty$ entonces, recordando que $\exists x\varphi \Rightarrow \varphi[D_n/x]\varphi \in \Gamma^\infty$. Asi que por MP, obtenemos $\Gamma^\infty \vdash \varphi[D_n/x]$ lo cual implica $\varphi[D_n/x] \in \Gamma^\infty$. Por hipótesis de inducción esto implica $\mathcal{M} \vDash \varphi[\iota_x \rightsquigarrow [D_n]]$.

$$\text{Como } [D_n] \in A \text{ esto implica que } \mathcal{M} \vDash \exists x\varphi$$

$\cdot\Gamma$ consistente, $\mathcal{L} \rightsquigarrow \mathcal{L}^* = \mathcal{L} \cup \{D_n | n \in \mathfrak{N}\}, \Gamma \rightsquigarrow \Gamma^\infty$ maximalmente en $\mathcal{L}^*$ consistente.
Para cada φ fórmula de $\mathcal{L}^*$:

$$\mathcal{M} \vDash \varphi[\iota] \text{ si y sólo si } \varphi \in \Gamma^{\infty}$$
$$\mathcal{B} = (A, \mathcal{R}_i, \mathcal{F}_i, c_i) \text{ es una } \mathcal{L}\text{-estructura.}$$

Para cada φ en $\mathcal{L}$:

$$\mathcal{B} \vDash \varphi[\iota] \text{ si y sólo si } \varphi \in \Gamma^{\infty}$$

En particular $\mathcal{B} \nvDash \varphi[\iota]$ para todo $\varphi \in \Gamma$.

$$\therefore \mathcal{B} \vDash \Gamma[\iota]$$

Capítulo 3

Computabilidad

3.1. Teoría de la computabilidad

Los principales contribuidores en la teoría de la computabilidad fueron *Alan Turing* mediante su *máquina de Turing*, *Kurt Gödel* mediante la *teoría de la recursión* y *Alonzo Church* mediante el $\lambda-$*cálculo.* Los tres conceptos son equivalentes y como producto arrojan funciones computables.

Definición 45. Una maquina de Turing consta de:

- un alfabeto, es decir, un conjunto finito L;
- un conjunto finito S de estados;
- una función de transición con dominio subconjunto de $(S \cup \{s_o\}) \times (L \cup \{*\})$ y codominio $S \times (L \cup \{*\}) \times \{<, >, -\}$.

Observación 46. Toda maquina de Turing especifica de manera única una función cuyo dominio y rango son subconjuntos del conjunto de sucesiones finitas de L.

Ejemplo 47. Consideremos

$$L = \{1\}, \qquad S = \{s_1, s_f\},$$

y la función de transición:

$$\begin{gathered}(s_0, 1, s_0, 1, >)\\ (s_0, *, s_1, *, <)\\ (s_1, 1, s_0, 1, <)\\ (s_1, *, s_f, 1, -);\end{gathered}$$

esta es la función sucesor en lenguaje unario.

Definición 48. Una función $f : A \longrightarrow \mathbb{N}^p$, con $p \in \mathbb{Z}^+$, es computable si existe un algoritmo tal que al recibir el input n, eventualmente termina si, y sólo si $n \in A$, en cuyo caso el output es $f(n)$.

Ejemplo 49. La función *busy beaver* no es computable:

- Fijamos el alfabeto $\{1\}$,
- para cada n,
$$A_n := \{f | f \in C\},$$
donde C es la colección de funciones computables por una máquina de Turing con a lo más n estados (más, quizá, el estado final).

$$A_1 := \{\varnothing, n \mapsto n+1, n \mapsto n, n \mapsto n-1, 0, 1\}.$$

Cada A_n es finito (tiene a lo sumo $2^{2(n+1)(n+1)\cdot 2\cdot 3}$ de elementos). La función **busy beaver** está dada por:

$$\sigma : \mathbb{N} \longrightarrow \mathbb{N}$$
$$\sigma(n) = \text{máx}\{f(0) | 0 \in \text{dom}(f), f \in A_n\}.$$

Definición 50. (máquina universal de Turing)
Llamamos así a la función $\Phi : A \longrightarrow \mathbb{N}$, con $A \subseteq \mathbb{N} \times \mathbb{N}$. $\Phi(e, n)$ es el resultado de introducir el imput n en la $e-$ésima máquina de Turing.

Por la tesis de Church-Turing, la función Φ es computable.

Observación 51. Para toda función computable f, existe e tal que

$$f(n) = \Phi(e, n).$$

Lema 52. *Para toda función computable f, existen una infinidad de e tales que*

$$f(n) = \Phi(e, n).$$

Definición 53. (función total)
Una función computable se dice total si para cualquier número natural como input, la función se detiene, es decir, el cálculo de su output se puede obtener en un número finito de pasos.

Teorema 54. *El conjunto*

$$e \in \mathbb{N} | \Phi(e, -) \text{ es una función total}$$

no es computable. Es decir, no existe un algoritmo que, dado e, decida si la $e-$ésima función computable es total.

Demostración: Supongamos que este conjunto si es computable, entonces hay una función computable

$$l \mapsto f_l,$$

la $l-$ésima función computable.

```
input(l);
j=0;
for(i=0;j<l;i++){
    if(Phi(i,-) es total)
        j++;
}return i;
```

Por lo tanto, la función

$$u : \mathbb{N}^2 \longrightarrow \mathbb{N}$$
$$(a, b) \mapsto f_a(b)$$

es total computable. Sea $g : \mathbb{N} \longrightarrow \mathbb{N}$, dada para cada $k \in \mathbb{N}$ por

$$g(k) = u(k, k) + 1.$$

Tenemos que g es computable, por lo tanto, existe l tal que $g = f_l$. Entonces,

$$u(l, l) = f_l(l) = g(l) = u(l, l) + 1,$$

lo cuál es una contradicción. □

Definición 55. (conjunto computable)
Sea $X \subseteq \mathbb{N}^k$. Decimos que X es computable si $\chi_X : \mathbb{N}^k \longrightarrow \{0, 1\}$ es una función total computable. Es decir, X es computable si existe un algoritmo con imput n, tal que determina si $n \in X$ o no.

3.2. Problema de la detención

También conocido por su nombre en alemán *entscheindungsproblem*, o en inglés *halting problem*.

Teorema 56. *(problema de la detención)*
El conjunto

$$\{(e,n)|\Phi(e,n) \text{ está definida}\}$$

no es computable. Es decir, no existe un algoritmo que diga cuáles algoritmos se detienen.

Demostración: Basta demostrar que $D = \{e \in \mathbb{N}|\Phi(e,e) \text{ está definido}\}$ no es computable. Supongamos que D es computable. Entonces, la función

$$\begin{aligned} f &: \mathbb{N} \longrightarrow \mathbb{N} \\ f(n) &= \begin{cases} 1 & n \notin D \\ \text{indefinido} & n \in D \end{cases} \end{aligned}$$

es computable. Está es representada por el siguiente pseudocodigo:

```
input(n)
if(chi_D(n)==0)
    return 1;
else
    for(i=1; ;i++)
```

Por lo tanto, sea e tal que

$$f = \Phi(e, -).$$

Entonces,

$$\begin{aligned} \Phi(e,e) &= f(e) \\ &= \begin{cases} 1 & e \notin D \\ \text{indefinido} & e \in D \end{cases} \\ &= \begin{cases} 1 & \Phi(e,e) \text{ no está definido} \\ \text{indefinido} & \Phi(e,e) \text{ está definido} \end{cases}, \end{aligned}$$

lo cuál es absurdo, luego D no es computable. □

Proposición 57. *Si X y Y son computables, entonces*

$$X \cup Y, \qquad X \cap Y, \qquad X \setminus Y$$

también lo son.

Demostración:

- $\chi_{X \cup Y} = \text{máx}\{\chi_X, \chi_Y\}$;
- $\chi_{X \cap Y} = \text{mín}\{\chi_X, \chi_Y\} = \chi_X \cdot \chi_Y$;
- $X \setminus Y = X \cap (\mathbb{N} \setminus Y)$, $\chi_{\mathbb{N} \setminus Y} = 1 - \chi_Y$, entonces $\chi_{X \setminus Y} = \chi_X \cdot (1 - \chi_Y)$.

□

Proposición 58. *Si $X \subseteq \mathbb{N}^2$ es computable, entonces:*

1. *Dado n,*
$$Y = \{m \in \mathbb{N} | \exists k < n((m,k) \in X)\}$$
es computable.

2. *En general,*
$$Z = \{m \in \mathbb{N} | \exists k((m,k) \in X)\}$$
no es computable.

Demostración:

1. La demostración se sigue del siguiente pseudocodigo:

```
input(m)
for(k=0;k<n;k++)
    if(Xx(m,k))
        return 1;
return 0;
```

2. Recordemos que
$$D = \{e | \Phi(e,e) \text{ está definido}\}$$
no es computable. Sea
$$X = \{(m,k) | \Phi(m,m) \text{ se detiene a lo más en } k \text{ pasos}\},$$
entonces X es computable. Note que:
$$\{m | \exists k((m,k) \in X)\} = \{m | \Phi(m,m) \text{ se detiene eventualmente}\} = D,$$
que no es computable.

□

Definición 59. (conjunto computablemente enumerable)
Un conjunto $X \subseteq \mathbb{N}$ es computablemente enumerable (también listable o c.e.) si existe una función total computable $f : \mathbb{N} \longrightarrow \mathcal{P}(\mathbb{N})$ tal que:

1. $f(0) = \varnothing$,
2. $\forall n(|f(n+1) \setminus f(n)| \leq 1 \wedge f(n) \subseteq f(n+1))$,
3. $X = \bigcup_{n \in \mathbb{N}} f(n)$.

Ejemplos 60.

1. $D = \{n | \Phi(n,n) \text{ está definido}\}$ es c.e.:

```
for(i=0; ;i++)
    for(j=0;j<=i;j++)
        correr Phi(j,j) i pasos
        si se detiene, print j;
```

2. $H = \{(a,b) | \Phi(a,b) \text{ está definido}\}$ es c.e.

Proposición 61. *Todo conjunto computable es c.e.*

Demostración: Si X es computable, entonces

```
for(n=0; ;n++)
    if(chi_X(n))
        print n;
```

□

Observación 62. El reciproco en general es falso, y el contraejemplo viene en los últimos ejemplos dados.

Proposición 63. *Si X y Y son computablemente enumerables, entonces:*

- *$X \cup Y$ también lo es,*
- *$X \cap Y$ también lo es.*

Demostración: Si A_X y A_Y son algoritmos que imprimen a X y Y respectivamente. Entonces,

```
for(n=1; ;n++)
    correr A_X n pasos
    correr A_Y n pasos
```

la cuál imprime a $X \cup Y$. Por otro lado,

```
for(n=1; ;n++)
    correr A_X n pasos, determinando L;
    correr A_Y n pasos, determinando L';
    print(L intersección L')
```

□

Teorema 64. *Para $X \subseteq \mathbb{N}$, los siguientes enunciados son equivalente:*

1. *X es c.e.*
2. *La función semicaracteristica de X, denotada σ_X, es computable. Esta función viene dada por la regla de asignación:*
$$\sigma_X(n) = \begin{cases} 1 \text{ si } n \in X \\ \uparrow \text{ en otro caso} \end{cases}.$$
3. *Existe una función computable f tal que* $\mathrm{dom}(f) = X$.
4. *Existe un $Y \subseteq \mathbb{N} \times \mathbb{N}$ computable tal que*
$$X = \{n \in \mathbb{N} | \exists k \in \mathbb{N}((n,k) \in Y)\}.$$

Demostración: 1. $\implies$ 2. :
Sea A_X el algoritmo que imprime a X. Entonces, el siguiente algoritmo calcula a σ_X:

```
input(n)
correr A_X
    if(se imprimió n)
        return 1;
```

2. $\Longrightarrow$ 3. :
Inmediato tomando $f = \sigma_X$.
3. $\Longrightarrow$ 4. :
Sea $Y = \{(n,k)|$el algoritmo que calcula $f(x)$ se detiene en, a lo más, k pasos$\}$. Notemos que Y es computable, con input (n,k). Se corre el algoritmo para $f(n)$ k pasos; si alcanza a detenerse, devuelve 1, en caso contrario devuelve 0. $\pi[Y] = X$, ya que $n \in X$ si, y sólo si $f(x)$ está definido si, y sólo si para algún k, el cálculo de $f(n)$ termina en k pasos si, y sólo si $(n,k) \in Y$.
4. $\Longrightarrow$. :
Supongamos Y como en la hipótesis, es decir, hay un algoritmo que calcula χ_Y, entonces el siguiente algoritmo imprime a X:

```
for(n=0; ;n++)
    sean a y b tales que n=2^a(2*b+1)
    if(chi__Y(a,b))
        print a;
```

□

Teorema 65. *(Kleene)*
Sea $X \subseteq \mathbb{N}$. Entonces,

$$X \text{ es computable} \Longleftrightarrow (X \text{ y } \mathbb{N} \setminus X \text{ son c.e.}).$$

Demostración:
$\Longrightarrow$): si X es computable, entonces $\mathbb{N} \setminus X$ también lo es. Luego, X y $\mathbb{N} \setminus X$ son c.e.
$\Longleftarrow$): sean A_X y $A_{\mathbb{N}\setminus X}$ los algoritmos que imprimen a X y $\mathbb{N} \setminus X$, respectivamente. El siguiente algoritmo calcula χ_X:

```
input(n)
correr A_X y A_{C(X)} en paralelo, esperar a que se imprima $n$
    si A_X lo imprimió
        return 1;
    si A_{C(X)} lo imprimió
        return 0;
```

□

Observación 66. Ni $\mathbb{N}\setminus D$, ni $\mathbb{N}^2\setminus H$ son c.e. (pues por el teorema de Kleene, D y H serían numerables).

Teorema 67. *Sea $f : A \longrightarrow \mathbb{N}$ tal que $A \subseteq \mathbb{N}$. Entonces,*

$$f \text{ es computable} \Longleftrightarrow Gr(f) \text{ es c.e.}$$

Demostración:
$\Longleftarrow$) Si $A_{Gr(f)}$ es el algoritmo que imprime a $Gr(f)$, entonces:

```
input(n)
correr A_{Gr(f)} tal que se imprima una pareja (x,y) tal que x=n
en tal caso, return y;
```

es un algoritmo que calcula a f.
$\Longrightarrow$) Considere el algoritmo:

```
for(n=0; ;n++)
    for(k=0;k<=;k++)
        corrar f(k) n pasos
        si se obtiene una respuesta y
            print(k,y);
```

□

Teorema 68. *Sea $X \subseteq \mathbb{N}$, con $X \neq \varnothing$. Entonces,*

$$X \text{ es c.e.} \iff (\exists f : \mathbb{N} \longrightarrow \mathbb{N})(f \text{ es total computable} \wedge X = \operatorname{ran}(f)).$$

Demostración:
$\Longleftarrow$) Si hay un algoritmo para f, entonces

```
for(n=0; ;n++)
    print f(n)
```

es un algoritmo que imprime $\operatorname{ran}(f) = X$.
$\Longrightarrow$) Sea $x_o \in X$ arbitrario. Considere el siguiente algoritmo:

```
input(n)
correr el algoritmo que imprime a X n pasos
    si no se imprime nada, return x_o;
    en caso contrario, si y es el último número impreso
    return y;
```

esta última función es total y $\operatorname{ran}(f) = X$. □

3.3. Computabilidad y lógica

Com parte del contexto historico, presentaremos programa de Hilbert, el cuál constaba de los siguientes puntos:

1. Precisar un lenguaje formal.
2. Codificar las reglas de la lógica.
3. Encontrar un conjunto de axiomas (que permita demostrar todo).
4. Demostrar que el sistema no admite contradicciones.

Sobre el tercer punto, en $\mathcal{L}_A = \{+, \cdot, \nvdash, S, <\}$, queremos un conjunto Σ de $\mathcal{L}_A-$fórmulas tales que:

1. $\mathbb{N} \vDash \Sigma$,
2. Σ es un conjunto computable,
3. $\{\varphi\ \mathcal{L}_A-\text{formulas} \mid \Sigma \vdash \varphi\}$ es computable;

pero es imposible que se cumplan las tres a la vez.

Observación 69. Sea $\mathcal{S}$ el conjunto de sucesiones finitas de símbolos de $\mathcal{L}$. Entonces, los siguientes conjuntos son computables:

$$\text{Term} = \{t \in \mathcal{S} | t \text{ es término}\},$$

$$\text{Form} = \{\varphi \in \mathcal{S} | \varphi \text{ es una fórmula}\},$$

$$\{(\varphi, x) \in \mathcal{S} \times \text{variables} | x \text{ es variable libre de } \varphi\},$$

$$\text{Sent} = \{\varphi \in \mathcal{S} | \varphi \text{ es un enunciado}\},$$

$$\{\varphi \in \mathcal{S} | \varphi \text{ es axioma lógico}\},$$

$$\{(a_1, a_2, a_3) \in \mathcal{S}^3 | a_1 \equiv a_2 \Rightarrow a_3 \vee a_2 \equiv a_1 \Rightarrow a_3\},$$

$$\text{Dem} = \{(\varphi_1, \ldots, \varphi_n) | (n \in \mathbb{N})(\varphi_i \text{ es fórmula}) \wedge (\varphi_1, \ldots, \varphi_n) \text{ es demostración formal}\}.$$

Por otro lado, el conjunto

$$\text{Teor} = \{\varphi \in \mathcal{S} | \text{ existe una demostración de } \varphi\}$$

no es computable, pero es computablemente enumerable.

Proposición 70. *Sea Σ un conjunto de $\mathcal{L}-$fórmulas.*

$$\textit{Dem}(\Sigma) = \{(\varphi_1, \ldots, \varphi_n) | (n \in \mathbb{N})(\varphi_i \textit{ es fórmula}) \wedge (\varphi_1, \ldots, \varphi_n) \textit{ es demostración a partir de } \Sigma\}.$$

Si Σ es computable, entonces Dem(Σ) también lo es. Por otro lado,

$$\begin{aligned} \textit{Teor} &= \{\varphi \in \mathcal{S} | \textit{existe una demostración de } \varphi \textit{ a partir de } \Sigma\} \\ &= \{\varphi \in \mathcal{S} | \Sigma \vdash \varphi\} \end{aligned}$$

es computablemente enumerable.

Definición 71. (completud)
Un conjunto de fórmulas Σ es completo si para cada fórmula φ,

$$\Sigma \vdash \varphi \vee \Sigma \vdash \neg\varphi.$$

Observación 72. Si Σ es completo y computable, entonces Teor(Σ) es computable. Si Σ es completo, entonces

$$\mathcal{S} \setminus \text{Teor}(\Sigma) = \{\varphi | \varphi \text{ no es fórmula } \vee \Sigma \vdash \neg\varphi\}.$$

Ejemplos 73.

- Todo conjunto inconsistente es completo.
- Si $\mathcal{L} = \{\leq\}$ y

$$\Sigma = \{x \leq x, x \leq y \implies y \leq x, (x \leq y \wedge y \in z) \implies x \leq y, x \leq y \wedge y \leq x,$$

$$\forall x \forall y (x < y \implies \exists z (x < z \wedge z < y)), \forall x \exists z z < x, \forall x \exists z x < z\}$$

  es completa.

- Si $\mathcal{L}_G = \{*, ()^{-1}, E\}$,

$$\Sigma = \{(x * y) * z = x * (y * z), x * E = x, x * (x)^{-1} = E,$$

$$\exists x \exists y \exists z (x \neq y \wedge y \neq z \wedge x \neq z \wedge \forall w (w = x \vee w = y \vee w = z))\}$$

  es completa.

- Dado p primo, la teoría de campos algebraicamente cerrados de caracteristica p, en el lenguaje $\mathcal{L}_A = \{+, \cdot, \nvdash, \nVdash\}$ es completo.
- La teoría de campos real-cerrados, en $\mathcal{L} = \{+, \cdot, \nvdash, \nVdash, \leq\}$ es completo.

Tenemos el siguiente hecho: *Ningún* Σ *en* $\mathcal{L}_A = \{+, \cdot, \nvdash, S, <\}$*, que sea un intento razonable de axiomatizar* $\mathbb{N}$*, puede ser completo y computable.*

- Observemos que $\mathrm{Th}(\mathbb{N}) = \{\varphi | \mathbb{N} \vDash \varphi\}$ no es computable.
- Cualquier $\Sigma \subseteq \{\varphi | \mathbb{N} \vDash \varphi\}$ computable.
- Cualquier Σ compatible en Aritmética de Robinson.
- $\Sigma =$Aritmética de Peano.

Definición 74. Un conjunto $X \subseteq \mathbb{N}^k$ es definible si existe una formula $\varphi(x_1, \ldots, x_k)$ tal que

$$X = \{(a_1, \ldots, a_k) | \mathbb{N} \vDash \varphi[a_1, \ldots, a_k]\}.$$

Ejemplos 75.

- Números pares:
$$\{n \in \mathbb{N} | \mathbb{N} \vDash \exists y(x = y + y)[n]\};$$
- Números primos:
$$\{n \in \mathbb{N} | \mathbb{N} \vdash \forall a \forall b(x = a * b \implies (a = \nVdash \vee b = \nVdash))[n]\}.$$

Teorema 76. *Si* $f : \mathbb{N} \longrightarrow \mathbb{N}$ *es parcialmente computable, entonces* $Gr(f) = \{(n, f(n)) | n \in \mathrm{dom}(f)\}$ *es definible.*

La demostración de este teorema requiere que desarrollemos algunas cosas difentes antes.

Observación 77. Señalaremos una nueva forma de códificar parejas:

$$(a, b) \text{ se codifica por } (a + b)^2 + a.$$

Dadp n, me fijo en la sucesión de cuadrados perfectos $(n^2)_{n \in \mathbb{N}}$, y encuentro m tal que $m^2 \leq n < (m + 1)^2$. Hacemos $a = n - m^2$, $b = m - a$, y n codifica (a, b). Notemos:

$$''n \text{ codifica } (a, b)'' \iff \mathbb{N} \vDash ((x + y) * (x + y) + x = z)[a, b, n].$$

Ahora, para codificar tuplas: Para cada i, $q_i = \text{código}(m_i, i)$. Sea

$$n = \text{máx}\{q_0, \ldots, q_{k-1}\},$$

y sea

$$u = \prod_{i<k} (1 + (q_i + 1)n!);$$

entonces,

$$(m_0, \ldots, m_{n-1}) \text{ se codifica como } (u + n!)^2 + u = \text{código}(u, n!).$$

Lema 78. *Si* $q < r < n$*, entonces* $(1 + (q + 1)n!)$ *es primo relativo con* $(1 + (r + 1)n!)$*.*

Demostración: Supongamos que p es primo y

$$p|1+(q+1)n! \wedge p|1+(r+1)n!,$$

entonces

$$p|n!(r-q)$$

pero $p \not| n!$ (en tal caso, $p=1$ lo cuál es absurdo), por lo tanto,

$$p|r-q.$$

Como $(p,i)=1$, para todo $i<n$ y $r-q<n$, esto es una contradicción. □

Lema 79. *Si $q<n$, entonces*

$$(1+(q+1)n!)|u \Longleftrightarrow q=q_j, \text{ para algún } j.$$

Demostración: Por el lema anterior, si $q<n$ es distinto de $q_0,\ldots,q_{k-1}$, entonces

$$(\forall l \in [[0,k-1]])((1+(q+1)n!, 1+(q_l+1)n!)=1).$$

Luego,

$$1+(q+1)n! \text{ es primo relativo con } \prod_{j<k}(1+(q_j+1)n!)=u.$$

□

Lema 80. *Para cada i, m_i es el mínimo m tal que*

$$1+(\textit{código}(m,i)-1)n!|u \tag{R}$$

Demostración: m_i sí satisface (R), falta ver que ningún $m<m_i$ satisface (R). Supongamos que $m<m_i$ satisface (R), entonces sea $q=$ código(m,i).Entonces, $(1+(q+1)n!)|u$, pero por el lema anterior

$$q=q_j, \text{ para algún } j.$$

Luego, $q=q_i$, es decir, $m=m_i$, lo cuál es absurdo. □

Observación 81. a es la $i-$ésima entrada de la tupla codificada por t.

$(\exists u<t \wedge \exists v<t)[(t$ códifica a $(u,v) \wedge \exists z(z$ codifica a $(a,i)) \wedge 1+(z+1)v|u)] \wedge$(además, a es el mínimo con esa propiedad).

$$(\exists a)\psi(a,i,t) \wedge \forall j>i \exists a \psi(a,i,t)$$

"la sucesión codificada por t es de longitud i".

Observación 82. T máquina de Turing.
Estados $\{s_i, s_f\}$.
$T=\{(0,s_i,1,s_i,>),(1,s_i,1,s_i,>),(*,s_i,*,s_f,-)\}$.
En cada paso, tenemos la foto instantanea

$$(i,j,k)$$

donde,

- i es el estado actual,
- j es la posición,
- k es lo que hay escrito en la cinta.

Así pues,

- $''(x,y,z)$ es una foto instantanea valida" $(FI(x,y,z))$
  $(x = \not{\vdash} \vee x = S\not{\vdash}) \wedge \forall u(\exists l(u < l \wedge \text{long}(l,z))(\exists l(\text{long}(l,z) \wedge y \leq l) \implies \exists v(\text{Tupla}(v,n,z) \implies (v = \not{\vdash} \vee v = S\not{\vdash} \vee v = SS\not{\vdash}))))$

- $''(x_1,y_1,z_1)$ es la foto instantanea inmeadiatamente posterior a $(x_2,y_2,z_2)''$ $T(x_1,y_1,z_1,x_2,y_2,z_2)$ $\big(''(x_1,y_1,z_1)$ es toto inst. valida$''\big) \wedge \exists s\,\big(\text{Tupla}(s,y_1,z_1) \wedge \big((s = \not{\vdash} \wedge x_2 = x_1) \vee (s = S\not{\vdash} \wedge x_2 = x_1) \wedge y_2 = y_1 + S\not{\vdash} \wedge \exists l\,\big(\text{long}(l,z_1) \wedge \text{long}(l,z_2) \wedge \forall u < l\,\big(u \neq y_1 \Rightarrow \exists t\,(\text{Tupla}(t,v,z_1) \wedge \cdots \wedge \text{Tupla}(t,u,z_2) \wedge (s = \not{\vdash} \wedge \text{Tupla}(S\not{\vdash},y_1,z_2) \vee s = S\not{\vdash} \wedge \text{Tupla}(S\not{\vdash},y_1,z_2) \vee (\text{long}(y_1,z_1) \wedge x_2 = S\not{\vdash} \wedge y_2 = y_1)$.

- "si yo corro T con input x, entonces la máquina se detiene y el output es y'' $C(x,y)$
  $\exists s\,\exists l\big(\text{long}(l,s) \wedge (\exists w)\,(l = SSS\not{\vdash}w \wedge \text{Tupla}(\not{\vdash},\not{\vdash},s) \wedge \text{Tupla}(\not{\vdash},S\not{\vdash},s) \wedge \text{Tupla}(x,SS\not{\vdash},s)$
  $\wedge \forall u < w\,(\exists x_1\exists y_1\exists z_1\exists x_2\exists y_2\exists z_2\; T(x_1,y_1,z_1,x_2,y_2,z_2) \wedge \text{Tupla}(x_1,z\cdot w,s) \wedge \text{Tupla}(y_1,Sz\cdot w,s) \wedge \text{Tupla}(z_1,SSz\cdot w,s) \wedge \text{Tupla}(x_2,SSSz\cdot w,s) \wedge \text{Tupla}(y_2,SSSSz\cdot w,s) \wedge \text{Tupla}(z_2,SSSSSz\cdot w,s)) \wedge \exists\alpha\, SSS\alpha = l \wedge \text{Tupla}(S\not{\vdash},\alpha,s) \wedge \text{Tupla}(y,SS\alpha,s)$

Demostración: (de 76)
Existen las siguientes formulas:

- $\text{Par}(x,y,z)$ que es $''z$ codifica $(x,y)''$.
- $\text{Tupla}(x,y,z)$ que es $''x$ es la $y-$ésima entrada de la tupla codificada por z''.
- $\text{Long}(y,z)$ que es $''$la longitud de la tupla codificada por z es y''.
- $\text{FI}(x,y,z)$.
- $T(x_1,y_1,z_1,x_2,y_2,z_2)$.
- $C(x,y)$.

Por lo que se tiene el teorema. □

Teorema 83. *(Indecibilidad de la Aritmética)*
El conjunto $Th(\mathbb{N}) = \{\mathbb{N} \vDash \varphi\}$ *no es computable.*

Demostración. Supongamos que sí es decidible, es decir, existe una enumeración efectiva de las formulas $\varphi_1,\ldots,\varphi_n,\ldots$ y un algoritmo que calcula correctamente

$$\chi_{\text{Th}(\mathbb{N})}(n) = \begin{cases} 1, & \text{si } \mathbb{N} \vDash \varphi_n \\ 0, & \text{si } \mathbb{N} \nvDash \varphi_n \end{cases} .$$

Por el teorema anterior,

$$\{(n,0) | \mathbb{N} \nvDash \varphi_n\} \cup \{(n,1) | \mathbb{N} \vDash \varphi_n\}$$

es definible, luego, existe una fórmula $\varphi(x,y)$ tal que

$$(\mathbb{N} \vDash \varphi[n,0] \iff \mathbb{N} \nvDash \varphi_n) \wedge (\mathbb{N} \vDash \varphi[n,1] \iff \mathbb{N} \vDash \varphi_n).$$

Sea $\psi(x) \equiv \varphi(x,S\not{\vdash})$. Entonces,

$$\mathbb{N} \vDash \psi[m] \iff \mathbb{N} \vDash \varphi_m.$$

Sea e tal que $\neg\psi$ es φ_e, entonces

$$\mathbb{N} \vDash \varphi_e[e] \iff \mathbb{N} \vDash \psi[e] \iff \mathbb{N} \vDash \neg\psi[e],$$

lo cuál es absurdo. □

Teorema 84. *(Indefinibilidad de la verdad de Tarski)*
Th(N) no es definible.

Demostración. Si fuera definible, entonces existiría

$$\mathbb{N} \vDash \psi[m] \Longleftrightarrow \mathbb{N} \vDash \varphi_m.$$

Sea e tal que $\neg\psi$ es φ_e, entonces

$$\mathbb{N} \vDash \varphi_e[e] \Longleftrightarrow \mathbb{N} \vDash \psi[e] \Longleftrightarrow \mathbb{N} \vDash \neg\psi[e],$$

lo cuál es absurdo. □

Corolario 85. *Si $\Sigma \subseteq Th(\mathbb{N})$ es computable, entonces es incompleto.*

Demostración. De lo contrario, si $\Sigma \subseteq \mathrm{Th}(\mathbb{N})$ es computable y compacto, entonces:

$$\mathrm{Teor}(\Sigma) = \{\varphi | \Sigma \vdash \varphi\}.$$

sería computable. Pero,

$$\mathrm{Th}(\mathbb{N}) \subseteq \mathrm{Teor}(\Sigma) \subseteq \mathrm{Th}(\mathbb{N});$$

ambas contenciones se tienen de los siguientes hechos:

- Si $\varphi \notin \mathrm{Teor}(\Sigma)$, entonces $\Sigma \nvdash \varphi$ por completud, $\Sigma \vdash \neg\varphi$ por lo tanto $\mathbb{N} \vDash \neg\varphi \implies \mathbb{N} \nvDash \varphi$.
- Por el teorema de correctud: $\mathbb{N} \vDash \Sigma$ así que si $\Sigma \vdash \varphi$, $\Sigma \vDash \varphi$, luego $\mathbb{N} \vDash \varphi$.

Pero, $\mathrm{Teor}(\Sigma)$ es computable y $\mathrm{Th}(\mathbb{N})$ no lo es, lo cuál es una contradicción. □

Definición 86. (Aritmética de Robinson)
El conjunto Q (RA) consta de las siguientes formulas:

1. $\forall x \ \neg(\not\vdash = Sx)$;
2. $\forall x \forall y \ (Sx = Sy \implies x = y)$;
3. $\forall x \ \neg(x < \not\vdash)$;
4. $\forall x \forall y \ (x < Sy \implies (x < y \vee x = y))$;
5. $\forall x \forall y \ (x < y \vee x = y \vee y < x)$;
6. $\forall x \ (x + \not\vdash = x)$;
7. $\forall x \forall y \ (x + Sy = S(x + y))$;
8. $\forall x \ (x \cdot \not\vdash = \not\vdash)$;
9. $\forall x \forall y \ (x \cdot Sy = x \cdot y + x)$.

Observación 87. Q es un conjunto finito.

Definición 88.

1. $\ltimes := \underbrace{S \ldots S}_{n-\text{veces}} \not\vdash$.
2. Un conjunto $X \subseteq \mathbb{N}^k$ es $\Sigma-$representable si existe una fórmula φ tal que:

$$(n_1, \ldots, n_k) \in X \Longleftrightarrow \Sigma \vdash \varphi[x_1/\ltimes_1, \ldots, x_k/\ltimes_k]$$

y

$$(n_1, \ldots, n_k) \notin X \Longleftrightarrow \Sigma \vdash \neg\varphi[x_1/\ltimes_1, \ldots, x_k/\ltimes_k].$$

3. Un conjunto $X \subseteq \mathbb{N}^k$ es débilmente $\Sigma-$representable si existe una fórmula φ tal que:

$$(n_1, \ldots, n_k) \in X \Longleftrightarrow \Sigma \vdash \varphi[x_1/\ltimes_1, \ldots, x_k/\ltimes_k].$$

Ejemplo 89. $X = \{n \in \mathbb{N} | n \text{ es par}\}$ es $Q-$representable. Sea φ la fórmula

$$\exists y \ \ (\not\vDash \cdot y = x),$$

siendo esta la fórmula requerida.

Teorema 90. *1. Toda función computable es $Q-$representable.*

2. Todo conjunto computable es $Q-$representable.

Demostración.

1. Análoga a la demostración de

$$\text{"}f \text{ es computable } \Longrightarrow \ f \text{ es definible"}.$$

2. Sea A computable y sea $\varphi(x, y)$ la fórmula que $Q-$representa a χ_A. Es decir,

$$(n, m) \in \mathrm{Gr}(\chi_A) \Longleftrightarrow Q \vdash \varphi[\ltimes/x, \gtrdot/y] \wedge (n, m) \notin \mathrm{Gr}(\chi_A) \Longleftrightarrow Q \vdash \neg\varphi[\ltimes/x, \gtrdot/y].$$

Sea $\psi(x) \equiv (y = S\not\vDash \wedge \varphi(x, y))$. Afirmamos que ψ representa a A:

$$\begin{aligned} n \in A &\Longleftrightarrow \chi_A(n) = 1 \\ &\Longleftrightarrow (n, 1) \in \mathrm{Gr}(\chi_A) \\ &\Longleftrightarrow Q \vdash \varphi[\ltimes/x, S\not\vDash/y] \\ &\Longleftrightarrow Q \vdash (y = S\not\vDash \wedge \varphi(x, y))[\ltimes/x]; \end{aligned}$$

y

$$\begin{aligned} n \notin A &\Longleftrightarrow \chi_A(n) = 0 \\ &\Longleftrightarrow (n, 1) \notin \mathrm{Gr}(\chi_A) \\ &\Longleftrightarrow Q \vdash \neg\varphi[\ltimes/x, \not\vDash/y] \\ &\Longleftrightarrow Q \vdash \neg(y = S\not\vDash \wedge \psi(x, y))[\mathbb{N}/x]. \end{aligned}$$

□

Teorema 91. *(Primer teorema de incompletitud de Gödel)*
Si Σ es cualquier conjunto de fórmulas compatible con Q, es decir, tal que $\Sigma \cup Q$ es consistente; entonces,

1. *Teor(Σ) no es conjunto computable;*

2. *si Σ es computable, entonces es incompleto.*

Demostración.

1. Probemos que $\mathrm{Teor}(\Sigma \cup Q)$ no es computable. Sea $\varphi_1(x), \varphi_2(x), \ldots, \varphi_n(x), \ldots$ una enumeración efectiva de las fórmulas con una variable libre. Sea

$$X = \{m \in \mathbb{N} | \Sigma \cup Q \vdash \varphi_m[\gtrdot/x]\}.$$

Si $\mathrm{Teor}(\Sigma \cup Q)$ fuera computable, X también lo sería, luego X sería $Q-$representable y X también sería $(\Sigma \cup Q)-$representable, es decir, existe una formila $\psi(x)$ tal que para cada $m \in \mathbb{N}$,

$$m \in X \Longleftrightarrow \Sigma \cup Q \vdash \psi[\gtrdot/x],$$

y

$$m \notin X \Longleftrightarrow \Sigma \cup Q \vdash \neg\psi[\gtrdot/x].$$

Sea e tal que $\neg\psi$ es la formula φ_e. Entonces,

$$\begin{aligned} e \in X &\Longleftrightarrow \Sigma \cup Q \vdash \varphi_e[/x] \quad \text{(definición de } X) \\ &\Longleftrightarrow \Sigma \cup Q \vdash \psi[/x] \quad \text{(definición de } \psi) \\ &\Longleftrightarrow \Sigma \cup Q \vdash \neg\varphi_e[/x], \end{aligned}$$

lo cuál es una contradicción. Por lo tanto, $e \notin X$, pero

$$\begin{aligned} e \notin X &\Longleftrightarrow \Sigma \cup Q \nvdash \varphi_e[/x] \quad \text{(definición de } X) \\ &\Longleftrightarrow \Sigma \cup Q \vdash \neg\psi[/x] \quad \text{(definición de } \psi) \\ &\Longleftrightarrow \Sigma \cup Q \vdash \varphi_e[/x], \end{aligned}$$

por lo tanto, $\mathrm{Teor}(\Sigma \cup Q)$ es no computable. Ahora, supongamos que $\mathrm{Teor}(\Sigma)$ es computable. Sea $\mathcal{O}$ la conjunción de todos los elementos de Q. Entonces, el conjunto $\{\varphi | \Sigma \vdash (\mathcal{O} \implies \varphi)\}$ también lo sería. Pero

$$\begin{aligned} \{\varphi | \Sigma \vdash (\mathcal{O} \implies \varphi)\} &= \{\varphi | \Sigma \cup \{\mathcal{O}\} \vdash \varphi\} \\ &= \{\varphi | \Sigma \cup Q \vdash \varphi\} \\ &= \mathrm{Teor}(\Sigma \cup Q), \end{aligned}$$

lo cuál es una contradicción.

2. φ_e como en el punto anterior no se puede demostrar.

□

Corolario 92. *El problema de la validez es indecidible, es decir, $\{\varphi | \vdash \varphi\}$ no es computable.*

3.4. Segundo teorema de incompletitud

Observación 93. Sea $\varphi_1, \varphi_2, \ldots, \varphi_n, \ldots$ una enumeración efectiva de los enunciados de $\mathcal{L}_A$. $\psi_1(x), \ldots, \psi_n(x)$ de las fórmulas. Dado un enunciado ψ denotamos por

$$\ulcorner \psi \urcorner = \ulcorner e \urcorner \Longleftrightarrow \psi \text{ es } \varphi_e,$$

donde $\ulcorner \psi \urcorner$ es el número de Gödel de ψ.

Lema 94. *Para cada fórmula $\theta(y)$, existe un enunciado η tal que*

$$Q \vdash \theta[\ulcorner \eta \urcorner/y] \Longleftrightarrow \eta.$$

Demostración. Dada la fórmula $\theta(y)$, consideraremos la función

$$m \mapsto \ulcorner \psi_m[\gtrdot/x] \urcorner.$$

Esta función es computable, luego es $Q-$representable, es decir, existe la fórmula $\psi(x, y)$ tal que

$$Q \vdash \psi[\gtrdot/x, \ltimes/y] \Longleftrightarrow n = \ulcorner \psi_m[\gtrdot/x] \urcorner,$$

y

$$Q \vdash \neg\psi[\gtrdot/x, \ltimes/y] \iff n \neq \ulcorner\psi_m[\gtrdot/x]\urcorner.$$

Definimos

$$\chi(x) \equiv \forall y(\psi(x,y) \implies \theta(y)).$$

Hacemos

$$\eta \equiv \chi[\ulcorner\chi\urcorner/x].$$

Entonces,

$$Q \vdash \psi[\ulcorner\chi\urcorner/x, y] \iff y = \ulcorner\chi[\ulcorner\chi\urcorner/x]\urcorner,$$

es decir,

$$Q \vdash \psi[\ulcorner\chi\urcorner/x, y] \iff y = \ulcorner\eta\urcorner. \tag{*}$$

Por otra parte,

$$\eta \equiv \forall y(\psi[\ulcorner\chi\urcorner/x, y] \implies \theta(y)). \tag{**}$$

De (*) y (**) tenemos que

$$Q \vdash \eta \iff \forall y(y = \ulcorner\eta\urcorner \implies \theta(y));$$

por lo tanto,

$$Q \vdash \eta \iff \theta[\ulcorner\eta\urcorner/y].$$

□

Definición 95. *(Aritmética de Peano)*

$$\mathrm{PA} = Q \cup \{(\varphi[\nvdash/x] \wedge \forall x(\varphi(x) \implies \varphi[Sx/x])) \implies \forall x\varphi(x) | \varphi(x) \text{ es fórmula}\}.$$

Observación 96. Sea Σ computable tal que $Q \subseteq \Sigma$. Consideramos

$$\mathrm{Teor}(\Sigma) = \{m | \Sigma \vdash \varphi_n\},$$

el cuál es computablemente enumerable. Por lo tanto, este conjunto es débilmente representable en Q, es decir, existe $\psi(x)$ tal que para cada $m \in \mathbb{N}$,

$$m \in \mathrm{Teor}(\Sigma) \iff Q \vdash \psi[\gtrdot/x].$$

Cabe recalcar que esta última $\psi(x)$ se puede entender por *"x es demostrable"*.

Definición 97. Definimos $\mathrm{Con}(\Sigma)$ como el enunciado $\neg\psi[\ulcorner\nvdash = S\nvdash\urcorner/x]$.

Teorema 98. *(Segundo teorema de incompletitud de Gödel)*
Si Σ es consistente, computable y PA$\subseteq \Sigma$, entonces

$$\Sigma \vdash Con(\Sigma).$$

Demostración. Le aplicamos el lema de la autorreferencia a $\neg\psi$, obtenemos η tal que

$$Q \vdash \eta \iff \neg\psi[\ulcorner\eta\urcorner/x].$$

Sea e tal que $\eta \equiv \varphi_e$ ($\ulcorner\eta\urcorner = e$). Supongamos que $\Sigma \vdash \eta$, es decir, $\Sigma \vdash \varphi_e$. Entonces, $e \in \mathrm{Teor}(\Sigma)$ lo que implica $Q \vdash \psi[/x]$. Por otra parte, $\Sigma \vdash \eta \iff \neg\psi[\ulcorner\eta\urcorner/x]$, $Q \vdash \psi[/x]$, por lo tanto:

$$\Sigma \vdash \neg\psi[\ulcorner\eta\urcorner/x] \wedge \Sigma \vdash \psi[/x],$$

$$\Sigma \vdash \psi[/x] \wedge \Sigma \vdash \psi[/x].$$

Esto contradice la consistencia de Σ. Luego,

$$\Sigma \nvdash \eta.$$

□

Observación 99. En el teorema anterior, la proposición η se puede entender como *"yo no soy demostrable"*.

Observación 100.

- $\mathbb{N} \vDash \eta$, $\mathbb{N} \vDash \mathrm{Con}(\Sigma) \implies \eta$;
- $\mathbb{N} \vDash \mathrm{Con}(\Sigma) \implies \neg\psi[/x]$.

Las principales aportaciones de Gödel fueron:

- $\mathrm{PA} \nvdash \eta$,
- $\mathrm{PA} \vdash \mathrm{Con}(\Sigma) \implies \eta$, por lo tanto

$$\Sigma \vdash \mathrm{Con}(\Sigma) \implies \eta \wedge \Sigma \nvdash \eta,$$

así, si $\Sigma \vdash \mathrm{Con}(\Sigma)$, por MP obtendríamos $\Sigma \vdash \eta$, lo que es una contradicción.